Evidentia specification v2.2

16 April 2026 19:22

This is Software Specification Template (SST) v2.2.1 – Generative Specification Standard – Canonical Edition.

Software Specification Template (SST) v2.2.1

Generative Specification Standard – Canonical Edition
14 April 2026
17:42
Universal Governance: Supra-MetaRules v5.6
0. PURPOSE, SCOPE, AND DETERMINISM
0.1 This document is the complete, self-contained specification standard for deterministic software generation.
0.2 No external document is required to interpret or implement this standard for P1–P12 execution. All governance required for build execution is embedded in §25.
0.3 The SPS will deterministically construct the system IF AND ONLY IF the spec fully determines the system. Any implementation choice not explicitly constrained by this spec + embedded governance SHALL cause deterministic build failure E-SPEC-UNDERDETERMINED.
0.4 Supra-MetaRules v5.6 §1.1 Truth Discipline applies: The system must not fabricate factual content. Any speculative statement must be explicitly marked as speculation. When uncertain, the system must state: “I cannot confirm this.”
1. LAYER DEFINITIONS
L0 Boundary Rows (System-Level)
L1 Spec Layer
L1 Governance Layer
L1 Prompt Layer
L1 Generation Layer
L1 Arbitration Layer
L1 File Layer
L1 Validation Layer
L1 Iteration Layer
L1 Commit Layer
L1 Provisioning Layer
L1 Deployment Layer
L1 Logging Layer
L1 Budget Layer
L1 Routing Layer
L1 Failure Handling
L1 UI Layer
L1 Validated Output Enforcement Layer
L1 Form Pathway Enforcement Layer
L1 Legal Rule Engine Layer
L1 LPSL Contract Layer
L1 Data Dictionary Layer
L1 Implementation Boundary Enforcement Layer
L1 Action Research Safety Layer
2. POSTGRESQL SCHEMA DEFINITIONS (DATABASE-READY, ALIGNED WITH ER MODEL, VALIDATION LAYER, AND LEGAL RULE ENGINE LAYER)
SQL

```
-- Matter
CREATE TABLE matter (
matter_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
title TEXT NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'archived')),
user_id UUID NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Evidence
CREATE TABLE evidence (
evidence_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
matter_id UUID NOT NULL REFERENCES matter(matter_id),
source_id UUID NOT NULL REFERENCES source(source_id),
content TEXT NOT NULL,
timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Finding
CREATE TABLE finding (
finding_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
evidence_id UUID NOT NULL REFERENCES evidence(evidence_id),
legal_rule_id TEXT NOT NULL,
significance TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Legal Artefact
CREATE TABLE legal_artefact (
```

```
artefact_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
finding_id UUID NOT NULL REFERENCES finding(finding_id),
document_type VARCHAR(50) NOT NULL,
content_hash TEXT UNIQUE NOT NULL,
validation_status VARCHAR(20) NOT NULL CHECK (validation_status IN ('provisional', 'validated', 'contested')),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Source
CREATE TABLE source (
source_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
provider_adapter_id UUID NOT NULL REFERENCES provider_adapter(adapter_id),
type VARCHAR(50) NOT NULL,
url TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Rule (from Legal Rule Engine)
CREATE TABLE legal_rule (
rule_id TEXT PRIMARY KEY,
source_type VARCHAR(50) NOT NULL CHECK (source_type IN ('statute', 'case_law', 'regulation')),
citation TEXT NOT NULL,
jurisdiction VARCHAR(100) NOT NULL,
condition_set JSONB NOT NULL,
conclusion TEXT NOT NULL,
precedence_level INTEGER NOT NULL,
effective_date DATE NOT NULL,
rule_version TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Provider Adapter
CREATE TABLE provider_adapter (
adapter_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
lpsl_definition_id UUID NOT NULL,
provider_name VARCHAR(100) NOT NULL,
base_url TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Iteration (from SST)
CREATE TABLE iteration (
iteration_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
build_run_id UUID NOT NULL,
iteration_no INTEGER NOT NULL,
started_at TIMESTAMPTZ NOT NULL,
ended_at TIMESTAMPTZ,
status VARCHAR(50) NOT NULL,
action_research_mode BOOLEAN NOT NULL DEFAULT FALSE
);
-- Repo Provision
CREATE TABLE repo_provision (
repo_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
build_run_id UUID NOT NULL,
github_repo_url TEXT NOT NULL UNIQUE,
repo_name TEXT NOT NULL UNIQUE,
visibility VARCHAR(10) NOT NULL CHECK (visibility IN ('private', 'public')),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Deploy Provision
CREATE TABLE deploy_provision (
deploy_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
repo_id UUID NOT NULL REFERENCES repo_provision(repo_id),
platform VARCHAR(20) NOT NULL CHECK (platform IN ('render')),
service_id TEXT NOT NULL,
live_url TEXT,
status VARCHAR(20) NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- LPSL Contract
CREATE TABLE lpsl_contract (
lpsl_definition_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
provider_name VARCHAR(100) NOT NULL UNIQUE,
capability_map JSONB NOT NULL,
request_schema JSONB NOT NULL,
response_schema JSONB NOT NULL,
auth_method VARCHAR(20) NOT NULL CHECK (auth_method IN ('api_key', 'oauth2', 'none')),
rate_limit JSONB NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3. L0 BOUNDARY ROWS (SYSTEM-LEVEL)

Columns: L0 Boundary | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations | C12 Execution Mode

L0 Boundary	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations	C12 Execution Mode
Execute Meta system	Execute Meta system	User provides Evidentia master specification, app-spec pack, and LPSL provider definitions to system	User initiates a full deterministic build cycle from specification ingestion through generation, validation, commit, provisioning, and deployment of the complete Evidentia ecosystem (backend services, frontend UI, provider adapters, Legal Workflow Process modules)	L0	controller.py	Valid Evidentia specification package provided → system returns build status, repository creation status, provisioned service status, and deployed application URL or structured failure report	main() → load_spec() → load_smr() → build_prompt() → generate_code() → write_files() → validate_system() → update_spec_if_needed() → provision_repo() → commit_changes() → provision_deploy() → deploy_system()	FastAPI 0.115.0. ASGI: uvicorn. DB: Postgres 16. Frontend: Next.js 14 App Router, React Server Components only. State: useState only. No client state libraries	SMR-2025-11-30#L0, LPSL-DEPLOY-001	["Flask", "Django", "Pages Router", "Express", "Redux", "MobX", "class components", "Railway"]	generate
Execute iterative correction cycle	Execute iterative correction cycle	User provides initial Evidentia specification package only; system self-corrects without requiring human re-entry each cycle	System repeatedly refines generation input using deterministic error feedback until convergence or termination threshold	L0	controller.py	Flawed but parsable Evidentia specification provided → system performs corrective iterations and returns convergence or structured failure	iteration_loop() → generate_candidate() → evaluate_candidate() → classify_failure() → update_spec() → next loop	Iteration controller must use for-loop with hard limit from spec. No while-true. No recursive calls	SMR-2025-11-30#L0-ITER	["recursive iteration", "unbounded loop"]	extend
Produce audit-ready result	Produce audit-ready result	User receives traceable outcome	System returns not only the executable Evidentia application but also a complete machine trace of validation, fault classes, governance checks (including LPSL and LWP compliance), provisioning events, and deployment outcome	L0	logging_service.py, report_builder.py	Build completes or fails → complete audit record returned	capture_run_context() → log_event() → persist_run_log() → build_run_report()	Audit logs: JSON Lines. Path: /var/log/sps/run_{build_run_id}.jsonl. Immutable after write	SMR-2025-11-30#L0-LOG	["stdout logs", "ephemeral logs"]	generate

4. L1 SPEC LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Ingest specification	Spec Layer	Ingest specification	User drops or pushes Evidentia master JSON + app-spec pack + LPSL definitions into governed input location	System reads specification from canonical path, confirms presence, parses schema, and prepares a normalized internal representation	P1	load_spec(), spec_schema.py	Valid Evidentia JSON spec package is accepted; malformed JSON is rejected with explicit parse error	read_file() → json.loads() → schema_validate() → normalize_spec()	Input path: /input/spec.json. Encoding: UTF-8. Max size: 10MB	SMR-2025-11-30#L1-SPEC	["YAML input", "stdin"]
Validate specification contract	Spec Layer	Validate specification contract	User expects invalid Evidentia specifications to fail early	System checks required fields, types, endpoint definitions, UI requirements,	P1	spec_validator.py	Missing required field (system_name, endpoints, or LPSL provider	validate_required_keys() → validate_types() → validate_nest	All fields defined in Data Dictionary Layer.	SMR-2025-11-30#L1-SPEC	["silent defaulting"]

				deployment directives, LPSL provider entries, LWP legal constraints, and governance placeholders before any generation call			definitions) causes deterministic rejection	ed_contracts() → validate_lwp_legal_fields()	Unknown fields → E-SPEC-UNKNOWN-FIELD		
Canonicalize specification	Spec Layer	Canonicalize specification	User submits variant ordering or formatting of Evidentia master spec or app specs	System transforms semantically equivalent specifications into canonical internal structure to support deterministic prompting and diff comparison	P1	spec_normalizer.py	Two semantically identical Evidentia specs with different field order produce the same canonical internal object	sort_keys() → standardize_defaults() → freeze_internal_spec() → enforce_lwp_canonical_form()	Canonical form: keys sorted ASCII. Decimals: 6 places. Timestamps: RFC3339 UTC	SMR-2025-11-30#L1-SPEC	["float approximation"]

5. L1 GOVERNANCE LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Retrieve active SMR	Governance Layer	Retrieve active SMR	User expects Meta to use current rules without hard-binding code to a rule version	System queries rule registry and loads active rule content dynamically as text plus metadata, while the code remains decoupled from rule version identifiers	P2	smr_service.py, smr_registry.py	System injects the current active SMR content into generation input even when registry version changes	connect_registry() → get_active_rule() → return_rule_payload()	Registry: Postgres table smr_registry. Query: SELECT content FROM smr_registry WHERE active=TRUE	SMR-2025-11-30#L1-GOV	["hardcoded rules"]
Validate SMR availability	Governance Layer	Validate SMR availability	User expects build to halt safely if governance is missing	If no active SMR is retrievable, generation is blocked and governance failure is recorded	P2	smr_service.py	Missing SMR causes build termination with governance_unavailable classification	load_smr() → null check → raise governance error	Exactly one SMR row may have active=TRUE	SMR-2025-11-30#L1-GOV	["multiple active SMR"]
Enforce governance policy	Governance Layer	Enforce governance policy	User expects all generated outputs (including auto-generated LLM adapters and LWP legal reasoning paths) to comply with rule constraints such as code-only response, deterministic construction, and LPSL/LWP compliance	System applies pre-generation, post-generation, and post-validation governance filters	P2 / P6	governance_filter.py, evaluator.py	Commentary, markdown fences, unsafe omissions, or disallowed constructs (including non-LPSL provider definitions or non-deterministic legal paths) are detected and classified	preflight_governance_check() → scan_candidate_output() → assert_policy_compliance() → enforce_lwp_determinism()	Scan: AST parse + regex blocklist from prohibited_implementations. No prose outside comments	SMR-2025-11-30#L1-GOV	["markdown", "narrative text"]

6. L1 PROMPT LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations

Assemble generation input	Prompt Layer	Assemble generation input	User expects prompt composition to be deterministic and complete	System merges normalized Evidentia master spec, app specs, active SMR, LPSL provider definitions, LWP legal reasoning contract, output contract, file contract, and correction constraints into a single generation package	P3	prompt_builder.py	Prompt contains spec, SMR, LPSL definitions, LWP constraints, correction constraints, required file manifest, and code-only contract	merge_spec() → merge_smr() → merge_lpsl() → merge_lwp() → merge_constraints() → build_prompt_payload()	Prompt template: SYSTEM_PROMPT constant. Temperature=0. Top_p=1. JSON mode enforced	SMR-2025-11-30#L1-PROMPT	["temperature>0"]
Preserve output contract	Prompt Layer	Preserve output contract	User expects generator to return buildable files only	System encodes a strict output contract defining filenames, file contents, import conventions, app entrypoint rules, forbidden content, and explicit LPSL adapter generation rules plus LWP legal reasoning enforcement	P3	prompt_contracts.py	Missing required file (e.g., provider adapter class, protocol engine, or LWP legal module) in generated response causes contract validation failure	build_output_contract() → embed_required_files() → embed_lwp_contract()	Output: single JSON object. Keys: "files". Each file: "path", "content". No other keys	SMR-2025-11-30#L1-PROMPT	["markdown fences"]

7. L1 GENERATION LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Generate code	Generation Layer	Generate code	System sends governed generation package (including LPSL and LWP) to model provider	LLM returns candidate code bundle (backend services, frontend, auto-generated adapters, legal workflow modules) under deterministic routing settings	P4	llm_interface.py	Non-empty code bundle is returned for valid generation call	route_model() → call_provider() → receive_payload() → extract_files()	Provider: OpenAI gpt-5. Model lock: SHA256 pinned. Retries: 0	SMR-2025-11-30#L1-GEN	["random sampling"]
Handle generation failure	Generation Layer	Handle generation failure	System detects API faults, empty payloads, timeout, malformed file bundle, contract breach, LPSL violation, or LWP non-determinism	Generation failure is classified before any file write	P4	llm_interface.py, failure_classifier.py	API timeout returns structured generation_timeout; empty response returns generation_empty	try_provider_call() → exception branch / empty branch / malformed branch / lwp_violation branch	Timeout: 120s. On failure: no write, immediate classify	SMR-2025-11-30#L1-GEN	["partial write"]
Support secondary validation model	Generation Layer	Support secondary validation model	System optionally submits candidate code (including generated adapters and LWP modules) to a second model for critique or contradiction detection	Secondary model does not replace primary build; it provides validation signals for arbitration	P13	validator_llm.py	Secondary validator flags schema omission, route mismatch, LPSL non-compliance, or LWP legal reasoning error in candidate bundle	send_candidate_for_review() → receive_validation_feedback()	Secondary model: OpenAI gpt-5-mini. Temp=0. No write permission	SMR-2025-11-30#L1-GEN	["secondary overwrite"]

8. L1 ARBITRATION LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations

Resolve multi-model conflict	Arbitration Layer	Resolve multi-model conflict	System receives primary code and optional secondary validation signal	Arbitration engine decides whether to accept candidate, reject candidate, or add deterministic correction constraints (including LPSL adapter fixes and LWP legal path corrections)	P13	arbitration_engine.py	If validator flags missing endpoint, LPSL violation, or LWP non-determinism and evaluator confirms it, candidate is rejected	compare_primary_secondary() → weigh_runtime_truth() → select_outcome() → enforce_lwp_arbitration()	Rule: Runtime truth > validator. If evaluator.py passes and validator flags, validator ignored. If evaluator fails, candidate rejected	SMR-2025-11-30#L1-ARB	["validator override"]

9. L1 FILE LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Reset output directory	File Layer	Reset output directory	System ensures clean file state before writing candidate	Previous generated Evidentia application directory is removed or archived according to policy	P5	file_writer.py	Old files do not persist into new candidate directory unless explicitly retained	prepare_output_dir() → clean_or_archive()	Root: /repo. On rebuild: rm -rf /repo/*. On extend: no delete	SMR-2025-11-30#L1-FILE	["partial clean"]
Write generated files	File Layer	Write generated files	Candidate file bundle (including auto-generated provider adapters and LWP legal modules) is written into repository structure	System writes app files, module initializers, config files, test fixtures, manifests, LPSL-generated adapters, and LWP deterministic reasoning modules as dictated by output contract	P5	file_writer.py	Required files exist in correct locations after write	write_bundle() → write_file() → verify_files_present()	Write mode: 0o644. Dirs: 0o755. Atomic write:.tmp then rename	SMR-2025-11-30#L1-FILE	["world writable"]
Protect repository integrity	File Layer	Protect repository integrity	System prevents path traversal or forbidden writes	Only approved repository subpaths may be written	P5	file_writer.py, path_guard.py	Attempted write outside approved root is blocked and logged	validate_relative_path() → reject_illegal_path()	Approved roots: ["app/", "api/", "lib/", "migrations/", "tests/"]. Reject: ["..", "/etc", "~"]	SMR-2025-11-30#L1-FILE	["absolute paths"]

10. L1 VALIDATION LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Syntax validation	Validation Layer	Syntax validation	System compiles or imports generated code	Candidate must parse and import without syntax error	P6	evaluator.py	Invalid syntax fails before route tests	compile_candidate() / import_module()	Python: ast.parse. JS/TS: tsc --noEmit. Fail = E-SYNTAX	SMR-2025-11-30#L1-VAL	["runtime syntax check"]
Dependency validation	Validation Layer	Dependency validation	System verifies required imports and runtime dependencies are satisfiable within target environment	Missing dependency or invalid import path (including LPSL adapters or LWP modules) is classified as runtime setup failure	P6	evaluator.py, environment_checker.py	Candidate importing undefined internal module fails with explicit dependency error	import_check() → exception capture	Check: importlib.util.find_spec for py. npm ls for js. No network	SMR-2025-11-30#L1-VAL	["dynamic install"]
Structural validation	Validation Layer	Structural validation	System inspects generated application object and	Candidate must expose expected FastAPI app, required route map, provider	P6	evaluator.py	Required endpoints are present with expected	inspect_app() → collect_routes() →	FastAPI: app.routes must match spec endpoints[].	SMR-2025-11-30#L1-VAL	["extra routes"]

			route declarations	registry, and LWP legal reasoning endpoints			methods and paths	compare_to_spec()	No extras allowed		
Behaviour validation	Validation Layer	Behaviour validation	System calls endpoints and validates runtime behaviour	Candidate routes must respond with valid status codes and payload schemas; LWP legal reasoning paths must produce deterministic Evidence → Finding → Legal Rule → Legal Significance output	P6	evaluator.py	GET /health returns expected JSON shape and literal/value types	TestClient(app) → request → response parse → schema compare	TestClient only. No external HTTP. Schemas: Pydantic v2	SMR-2025-11-30#L1-VAL	["live network"]
UI validation	Validation Layer	UI validation	System checks generated UI files against declared screens, components, and essential interactions	UI contract is evaluated statically and, where applicable, via rendered DOM inspection for matter list, evidence viewer, finding panel, legal artefact viewer, protocol panel, and LWP status dashboard	P6	ui_evaluator.py	Required screen elements such as upload area, build status panel, fault log panel, and legal evidence viewer are present	load_ui_manifest() → parse_component_tree() → assert_required_elements()	Next.js: parse app/ directory. RSC only. No "use client" unless declared in C9	SMR-2025-11-30#L1-VAL	["client components by default"]
Governance validation	Validation Layer	Governance validation	System checks candidate output for rule breaches	Output must be code only, no markdown fences, no narrative contamination, no disallowed heuristic traces, and must enforce LWP deterministic legal reasoning	P6	evaluator.py, governance_filter.py	Candidate containing prose commentary or non-deterministic legal paths is rejected as governance violation	scan_text() → detect_non_code_segments() → detect_lwp_nondeterminism()	Prose detector: blocklist ["In this section", "As we can see"]. LWP: must call rule_applicator.py	SMR-2025-11-30#L1-VAL	["explanatory comments"]
Security validation	Validation Layer	Security validation	System checks for forbidden operations, unsafe shell execution, secret leakage, and uncontrolled network calls	Build must conform to project execution boundaries and LWP legal data isolation rules	P6	security_evaluator.py	Candidate containing plaintext API key write or unrestricted subprocess invocation is rejected	scan_ast() → detect_forbidden_calls()	Blocklist: ["subprocess", "os.system", "eval", "exec", "requests"]. Allow: ["httpx", "pg"] via DI only	SMR-2025-11-30#L1-VAL	["shell=True"]
Failure classification	Validation Layer	Failure classification	System maps faults into deterministic taxonomy	Each failure is categorized as syntax, dependency, structure, schema, runtime, governance, security, deployment, budget, lpsl_violation, or lwp_legal_reasoning_failure	P6	failure_classifier.py	Same fault pattern always returns same class code	classify_error() → return_failure_taxonomy()	Taxonomy: E-{LAYER}-{CODE}. E-VAL-SYNTAX, etc. No free text	SMR-2025-11-30#L1-VAL	["uncategorized error"]
Pattern detection across failures (Action Research)	Validation Layer	Pattern detection across failures (Action Research)	When Action_Research_Mode = ON, system identifies recurring failure patterns and maps them to governing variables	Recurring schema mismatches or LWP legal reasoning failures across iterations flagged as governing variable	P6	evaluator.py, action_research_analyzer.py	Recurring failure pattern with mode ON → governing variable identified	detect_cross_iteration_patterns() → map_to_governing_variable()	Governing var must be field in spec_updater.py. No new fields invented	SMR-2025-11-30#L1-VAL	["spec drift"]

Produce validation report	Validation Layer	Produce validation report	System emits machine-readable result package	Report includes pass/fail state, matched routes, missing routes, schema mismatches, governance/security findings, LPSL status, and LWP legal reasoning validation results	P6	validation_report.py	Report produced for every iteration regardless of pass or fail	assemble_report() → serialize_report()	Report: /reports/validation_{iteration}.json. Schema: validation_report.schema.json	SMR-2025-11-30#L1-VAL	["human readable only"]

11. L1 ITERATION LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations | C12 AR Safety Constraint

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations	C12 AR Safety Constraint
Update specification deterministically	Iteration Layer	Update specification deterministically	System transforms validation failures into correction constraints	Errors are appended to or merged into the live build spec without rewriting original intent fields arbitrarily; LWP legal reasoning corrections are preserved	P7	spec_updater.py	Schema mismatch or LWP legal error generates explicit corrective rule in updated spec	parse_validation_report() → derive_constraints() → append_constraints()	Constraints appended to spec.correction_constraints[]. Never modify spec.endpoints[]	SMR-2025-11-30#L1-ITER	["overwrite original"]	AR_MAX_CONSTRAINTS_PER_RUN=3
Action Research Cycle	Iteration Layer	Action Research Cycle	When Action_Research_Mode = ON: execute full cycle (1. Governing Variable Identification, 2. Hypothesis Formation (rule-based), 3. Micro-Intervention, 4. Observation, 5. Reflection, 6. Governing Variable Revision, 7. Consolidation/Retraction, 8. Hierarchical Validation). When OFF: skip all AR stages	derive_ar_cycle() with explicit mode branching; hypothesis generation is rule-based only; no stochastic variation permitted; LWP legal paths are validated in hierarchical step (LWP paths remain deterministic within each iteration; AR adaptation is controlled and does not violate single-iteration determinism)	P7	spec_updater.py, action_research_engine.py	Mode ON + failure pattern → full deterministic AR cycle; Mode OFF → no AR fields or stages	if action_research_mode == ON then execute AR stages else skip	AR state persisted in iteration.ar_state. Micro-intervention = one constraint only	SMR-2025-11-30#L1-ITER	["multi-change per iteration"]	AR_HARD_FREEZE_AFTER_ITER=5; AR_ROLLBACK_ON_DIVERGENCE=TRUE
Preserve correction history	Iteration Layer	Preserve correction history	System records iterative amendments	Each correction is versioned by iteration so later analysis can inspect convergence behaviour	P7	spec_updater.py, iteration_store.py	Iteration 2 contains both original spec and accumulated deterministic constraints	snapshot_spec() → persist_iteration_state()	Store: Postgres table iteration_spec_snapshot. Immutable	SMR-2025-11-30#L1-ITER	["mutable history"]	AR_DIFF_MUST_BE_ATOMIC
Control iteration loop	Iteration Layer	Control iteration loop	System repeats cycle under termination rules	Loop continues until valid system is produced or hard limit is reached	P7	controller.py	Candidate converges within limit or exits with structured failure report	for iteration in range(max_iterations)	max_iterations from spec.limits.max_iterations. Integer 1-20	SMR-2025-11-30#L1-ITER	["while true"]	AR_TERMINATE_ON_SPEC_GROWTH>10%
Terminate on limit	Iteration Layer	Terminate on limit	System stops after configured maximum iterations	Failure report includes all prior attempts and unresolved defects	P7	controller.py	Max iteration reached returns status failed_max	if iteration == limit: terminate_with_report()	Report must include iteration[].validation_report	SMR-2025-11-30#L1-ITER	["silent exit"]	AR_AUTO_ROLLBACK_TO_LAST_PASS

							_iterations					

12. L1 COMMIT LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Commit generated code	Commit Layer	Commit generated code	System stages and commits repository changes	Successful candidate is committed with deterministic message structure tied to build run metadata	P8	git_service.py	Repo shows new commit after successful validation	git_add() → git_commit()	Message: "sps: build {build_run_id} iter {n} mode {mode}". Author: sps-bot	SMR-2025-11-30#L1-COMMIT	["human author"]
Push to remote repository	Commit Layer	Push to remote repository	System updates remote repository	Commit is pushed to configured remote branch for deployment trigger	P8	git_service.py	Remote branch receives commit	git_push()	Remote: origin. Branch: main. Force: false	SMR-2025-11-30#L1-COMMIT	["force push"]

13. L1 PROVISIONING LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Create repository	Provisioning Layer	Create repository	System creates isolated GitHub repository for build	System calls GitHub API to create new private repo with name sps-build-{build_run_id}. No reuse. No manual step	P8.1	repo_provisioner.py	Repo does not exist → API call → repo exists with initial commit	create_repo() → verify_repo() → record_repo_provision()	GitHub API v		

Well?

Today

Continuing Software Specification Template (SST) v2.2.1 – Canonical Edition. No prompting required. Completing all remaining sections now per SMR v5.6 §2.3.1 and §2.4.1.

13. L1 PROVISIONING LAYER [CONTINUED]
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Create repository	Provisioning Layer	Create repository	System creates isolated GitHub repository for build	System calls GitHub API to create new private repo with name sps-build-{build_run_id}. No reuse. No manual step	P8.1	repo_provisioner.py	Repo does not exist → API call → repo exists with initial commit	create_repo() → verify_repo() → record_repo_provision()	GitHub API v3. PAT from spec.provisioning.github_token. Visibility: private	SMR-2025-11-30#L1-PROV	["manual repo", "shared repo"]
Inject environment secrets	Provisioning Layer	Inject environment secrets	System injects build-time secrets into repo/Render	Secrets from spec.provisioning.env are set via GitHub Encrypted Secrets and Render Env Groups. No plaintext	P8.2	secret_injector.py	Secret FOO missing → inject → Render service has FOO	encrypt_secret() → put_github_secret() → put_render_env()	Render API. GitHub Secrets API. AES-256	SMR-2025-11-30#L1-PROV	["plaintext env"]

14. L1 DEPLOYMENT LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System	C4 Full Deterministic	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementati	C10 Governan	C11 Prohibited

			Interaction	Description					on Boundary	ce Hash	Implementations
Provision Render service	Deployment Layer	Provision Render service	System creates Render Web Service from repo	Calls Render API: create service, link to repo_provision.repo_id, set buildCommand, startCommand, plan from spec.deploy.render	P9	render_provisioner.py	Service does not exist → API call → service exists in Render dashboard	create_render_service() → poll_status() → record_deploy_provision()	Render API v1. Auth: Bearer spec.provisioning.render_api_key. Plan: starter	SMR-2025-11-30#L1-DEPLOY	["Railway", "manual click"]
Trigger deployment	Deployment Layer	Trigger deployment	Repository push triggers Render build	Render auto-deploys on commit to main. System polls until live	P9	deploy.py, render_service.py	Commit pushed → Render build starts → live URL returned	trigger_deploy() → poll_status() → return_url()	Poll: GET /v1/services/{id}/deploys. Timeout: 600s	SMR-2025-11-30#L1-DEPLOY	["manual deploy"]
Verify deployment health	Deployment Layer	Verify deployment health	System calls live endpoint after deployment	Live URL must satisfy health and representative endpoint checks, including LWP legal reasoning test calls	P9	deploy.py, runtime_probe.py	Deployed /health returns expected payload	probe_live_url() → validate_live_response()	Probe: GET {url}/health. Timeout 10s. Expected: {"status":"ok"}	SMR-2025-11-30#L1-DEPLOY	["skip healthcheck"]

15. L1 LOGGING LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Log every build action	Logging Layer	Log every build action	System records every major state transition	Logs include time, iteration, action name, inputs, outputs, status, and error class	P10	logging_service.py	Persistent logs exist for each run and each iteration	capture_event() → persist_log()	Format: JSONL. Fields: ts,run_id,iter,action,status,meta	SMR-2025-11-30#L1-LOG	["printf"]
Log faults with forensic detail	Logging Layer	Log faults with forensic detail	System captures diagnostic information sufficient for replay and audit	Fault log includes stack trace summary, validation diffs, candidate metadata, governing spec snapshot, and LWP legal reasoning trace	P10	fault_logger.py	Fault entries are appended in JSON lines format	append_fault_log()	Stack trace: 10 frames max. No locals. Spec snapshot: hash only	SMR-2025-11-30#L1-LOG	["full dump"]
Produce run summary	Logging Layer	Produce run summary	System aggregates run outcome	Summary contains convergence state, iteration count, route coverage, deployment result, elapsed time, and LWP legal validation summary	P10	report_builder.py	End-of-run summary can be displayed in UI and saved to file	build_summary()	Path: /reports/run_{id}_summary.json	SMR-2025-11-30#L1-LOG	["text summary"]

16. L1 BUDGET LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations | C12 Binding

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations	C12 Binding
Estimate call cost	Budget Layer	Estimate call cost	System computes projected provider cost before invocation	Estimated token and provider cost are compared against active policy thresholds	P11	budget_service.py	Over-budget request is blocked before provider call	estimate_cost() → compare_budget()	Cost model: tokens*rate from spec.budget.rates. Currency: USD	SMR-2025-11-30#L1-BUDGET	["post-hoc billing"]	Binds to P4, P12
Enforce budget policy	Budget Layer	Enforce budget policy	System allows or denies generation call	Budget decisions are logged and surfaced to user interface. Exceeding budget	P11	budget_service.py	Blocked build records budget_exceeded without attempting API call	allow_or_block()	Limit: spec.budget.max_usd_per_run. Hard stop	SMR-2025-11-30#L1-BUDGET	["soft limit"]	Binds to P7, P4

				terminates iteration loop								
Bind budget to routing	Budget Layer	Bind budget to routing	System selects cheaper provider if budget remaining < 20%	If projected_cost > budget_remaining*0.2, route to spec.routing.secondary	P11/P12	budget_router.py	Low budget → secondary provider selected	check_budget() → reroute()	Secondary must be defined in spec.routing.secondary	SMR-2025-11-30#L1-BUDGET	["ignore budget"]	Binds to P12
Bind budget to iteration	Budget Layer	Bind budget to iteration	System terminates iteration loop if cumulative cost > max_usd_per_run	On each iteration, cumulative cost checked before generation	P11/P7	budget_iteration_guard.py	Iteration 3 exceeds budget → E-BUDGET-EXCEEDED, loop terminates	accumulate_cost() → check_limit() → terminate_loop()	Hard fail, no partial generation	SMR-2025-11-30#L1-BUDGET	["continue on overrun"]	Binds to P7

17. L1 ROUTING LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations | C12 Budget Binding

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations	C12 Budget Binding
Select model provider	Routing Layer	Select model provider	System chooses active generation provider by policy and budget	Default route uses OpenAI but architecture supports alternate providers. Budget layer may override to secondary	P12	routing_service.py	Policy selects configured primary provider correctly; low budget selects secondary	read_routing_policy() → check_budget() → choose_provider()	Policy: spec.routing.primary. Secondary: spec.routing.secondary	SMR-2025-11-30#L1-ROUT	["round robin"]	Budget binding active
Apply deterministic generation settings	Routing Layer	Apply deterministic generation settings	System enforces fixed generation parameters	Temperature, top-p, max tokens, retry rules, and provider-specific settings are fixed by policy	P12	routing_service.py	Same policy yields same settings bundle across runs	build_model_config()	Temp=0. Top_p=1. Seed=spec.routing.seed. Retries=0	SMR-2025-11-30#L1-ROUT	["temp>0"]	Budget binding: max_tokens reduced if budget<10%

18. L1 FAILURE HANDLING
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Return structured failure outcome	Failure Handling	Return structured failure outcome	User receives explicit machine-readable failure rather than ambiguous partial success	System exits with failure code, taxonomy, unresolved constraints, run trace, and LWP legal reasoning diagnostics	P7 / P10	controller.py, report_builder.py	Failure response includes diagnostics and no false success state	terminate_with_report()	Exit code: 1. Report: /reports/failure_{run_id}.json	SMR-2025-11-30#L1-FAIL	["exit 0 on fail"]

19. L1 UI LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Provide specification upload interface	UI Layer	Provide specification upload interface	User uploads or selects specification for build	UI offers validated input control for spec selection and build initiation	External Interface	frontend/spec_upload.tsx	User can submit a spec file and receive immediate	UI event → API /spec/validate	Next.js App Router. Component: Server Component.	SMR-2025-11-30#L1-UI	["drag-drop lib"]

							validation feedback		Upload: <input type=file>		
Provide project registry	UI Layer	Provide project registry	User views all builds and repos	UI lists all build_run_id, repo_url, deploy_url, status. Supports filter by status	External Interface	frontend/registry.tsx	Registry shows 3 builds with distinct repos	UI → /api/runs	Table: TanStack Table. Read-only	SMR-2025-11-30#L1-UI	["single project view"]
Provide build status dashboard	UI Layer	Provide build status dashboard	User monitors current build run	UI presents iteration count, stage, pass/fail status, provider route, budget state, deployment URL, LWP legal validation status, repo link	External Interface	frontend/dashboard.tsx	Build progress is visible and updates by run state	UI poll → /api/runs/{id}	Poll: 2s interval. No websocket. Read-only	SMR-2025-11-30#L1-UI	["websocket push"]
Provide fault inspection panel	UI Layer	Provide fault inspection panel	User inspects validation failures	UI exposes categorized faults, route mismatches, schema diffs, iteration history, and LWP legal reasoning errors	External Interface	frontend/fault_panel.tsx	User can open a fault and inspect forensic detail	UI → /api/runs/{id}/faults	Display: fault.code, fault.diff. No stack trace in UI	SMR-2025-11-30#L1-UI	["stack trace render"]
Provide repository /deployment panel	UI Layer	Provide repository /deployment panel	User sees commit and live deployment linkage	UI shows branch, commit hash, deployment status, environment name, live URL, Render service link	External Interface	frontend/deploy_panel.tsx	Deployment panel shows successful live URL after pass	UI → /api/runs/{id}/deployment	Link: <a href={url} target=_blank>	SMR-2025-11-30#L1-UI	["iframe embed"]
Provide parallel build visibility	UI Layer	Provide parallel build visibility	User sees concurrent builds	UI shows up to N active builds with status. N from spec.ui.max_parallel_display	External Interface	frontend/parallel.tsx	3 builds running → 3 cards visible	UI → /api/runs?status=active	N=3 default. SSE for updates	SMR-2025-11-30#L1-UI	["blocking UI"]

20. L1 VALIDATED OUTPUT ENFORCEMENT LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Enforce claim-to-evidence-to-source linkage	Validated Output Enforcement Layer	Enforce claim-to-evidence-to-source linkage	System ensures every legal artefact contains explicit claim linked to at least one Evidence item and each Evidence item linked to at least one Source	Every claim must reference ≥1 Evidence item and each Evidence item must reference ≥1 Source; missing linkage classified as output_incomplete	P6	output_enforcer.py	Generated artefact missing claim-to-evidence-to-source linkage is rejected	enforce_claim_evidence_source_link() → validate_traceability()	DB constraint: FOREIGN KEY. No nulls. Enforce at write	SMR-2025-11-30#L1-VOE	["orphan claims"]
Aggregate reasoning to conclusion	Validated Output Enforcement Layer	Aggregate reasoning to conclusion	System aggregates LWP reasoning paths into a single deterministic conclusion	All Evidence → Finding → Legal Rule → Legal Significance paths are aggregated into one conclusion section with validation_status field (Conclusion defined strictly as deterministic synthesis of evidence and LWP paths only; must not assert entitlement, liability, outcome, or prediction per Locked Workflow non-advisory rule)	P6	output_enforcer.py	Conclusion section contains aggregated reasoning and validation_status	aggregate_lwp_paths() → append_conclusion() (must enforce non-advisory constraint on conclusion text)	Conclusion text: template "Based on {n} evidence items and {m} rules, significance: {text}". Blocklist: ["entitled", "liable", "will win"]	SMR-2025-11-30#L1-VOE	["advisory language"]
Enforce output completeness and	Validated Output Enforce	Enforce output completeness and	System verifies every final artefact contains	Output artefact must include claim, evidence, source, reasoning, conclusion,	P6	output_enforcer.py	Artefact contains all required sections and	validate_output_structure() → set_validatio	JSON Schema: legal_artefact.schema.json.	SMR-2025-11-30#L1-VOE	["missing validation_status"]

validation status	ment Layer	validation status	required structure and explicit validation status	and validation_status fields (provisional, validated, contested); missing fields cause rejection (Conclusion must remain non-advisory; validation gate rejects any output violating Locked Workflow prohibition on legal determinations)			validation_status is one of provisional/validated/contested	n_status() (must enforce non-advisory constraint on conclusion text)	Required: all 6 fields		

21. L1 FORM PATHWAY ENFORCEMENT LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Identify form pathway	Form Pathway Enforcement Layer	Identify form pathway	System identifies the procedural form pathway for Tier 1 output based on matter classification and applicable law map	System determines candidate forms, relevant authority, procedural stage, filing context, and mandatory information still missing (Form pathway identification occurs only after applicable law map; final output is subject to validation enforcement gate)	P6	form_pathway_engine.py	Form pathway shortlist is generated and visible in Tier 1 output	identify_form_pathway() → validate_against_law_map() (sequencing must follow Locked Workflow order: Law Map → Form Pathway → Validation)	Lookup: legal_rule.condition_set → form_map table. No LLM	SMR-2025-11-30#L1-FPE	["LLM form choice"]
Generate draft form artefact	Form Pathway Enforcement Layer	Generate draft form artefact	System produces the draft form for professional review	Draft form is generated with all required fields pre-populated from evidence and factual record (Form generation occurs only after law map and is subject to final validation enforcement)	P6	form_generator.py	Draft form artefact is produced and contains all mandatory sections	generate_draft_form() → populate_from_evidence() (sequencing must follow Locked Workflow order: Law Map → Form Pathway → Validation)	Template engine: Jinja2. Templates: /forms/{jurisdiction}/{form_id}.pdf.j2	SMR-2025-11-30#L1-FPE	["fillable PDF"]
Enforce form completeness in validated output	Form Pathway Enforcement Layer	Enforce form completeness in validated output	System verifies every Tier 1 output contains the completed form pathway	Output artefact must include identified form pathway, draft form, and missing-information report (Form output must pass validation gate after generation per Locked Workflow sequence)	P6	form_pathway_enforcer.py	Tier 1 output contains form pathway, draft form, and completeness validation	enforce_form_completeness() → validate_form_pathway_in_output() (sequencing must follow Locked Workflow order: Law Map → Form Pathway → Validation)	Report: missing_fields[]. Empty list = complete	SMR-2025-11-30#L1-FPE	["partial form"]

22. L1 LEGAL RULE ENGINE LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Retrieve structured legal rules	Legal Rule Engine Layer	Retrieve structured legal rules	System queries the legal knowledge base for candidate rules based on matter classification	System returns versioned, structured Rule objects (rule_id, source_type, citation, condition_set, conclusion, precedence_level,	P3 / P6	rule_retriever.py	Candidate rules are returned with complete provenance and no fabrication	retrieve_rules() → filter_by_jurisdiction() → return_structured_rule_set()	Source: legal_rule table. No external API. effective_date <= today	SMR-2025-11-30#L1-LRE	["web search"]

			and jurisdiction	effective_date) bound to the current jurisdiction							
Match evidence to rules	Legal Rule Engine Layer	Match evidence to rules	System performs deterministic condition matching between Evidence and Rule condition_set	Each Evidence item is evaluated against Rule conditions; matches produce traceable Finding records with explicit rule_id linkage	P6	rule_matcher.py	Every matched rule produces exactly one Finding with verifiable condition satisfaction	evaluate_conditions() → generate_finding() → enforce_one_to_one_mapping()	Match: JSONLogic eval. No fuzzy match. All conditions boolean	SMR-2025-11-30#L1-LRE	["partial match"]
Apply rules and resolve precedence	Legal Rule Engine Layer	Apply rules and resolve precedence	System applies triggered rules, resolving conflicts according to explicit precedence logic	Rules are executed in precedence order (statute > case law, higher court > lower court, later > earlier); output is a single Legal Significance per Evidence path	P6	rule_applicator.py	Conflicting rules are resolved deterministically and logged with precedence trace	apply_precedence() → compute_significance() → log_resolution_path()	Order: source_type DESC, precedence_level DESC, effective_date DESC	SMR-2025-11-30#L1-LRE	["random tie-break"]
Enforce rule integrity in validated output	Legal Rule Engine Layer	Enforce rule integrity in validated output	System verifies that every generated artefact references only structured, traceable rules	Output artefact must contain rule provenance for every Finding and Legal Significance; missing or unstructured rules classified as rule_integrity_failure	P6	rule_integrity_enforcer.py	Artefact missing rule provenance or using unstructured rules is rejected	validate_rule_provenance() → enforce_integrity()	Provenance: finding.legal_rule_id must exist in legal_rule	SMR-2025-11-30#L1-LRE	["unstructured citation"]

23. L1 LPSL CONTRACT LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Define LPSL contract	LPSL Contract Layer	Define LPSL contract	System loads LPSL contract for each provider_adapter	Contract defines capability_map, request_schema, response_schema, auth_method, rate_limit. All required	P3	lpsl_loader.py	Missing request_schema → E-LPSL-INCOMPLETE	load_lpsl() → validate_schema()	File: /lpsl/{provider}.json. Schema: lpsl.schema.json	SMR-2025-11-30#L1-LPSL	["undocumented provider"]
Generate adapter from LPSL	LPSL Contract Layer	Generate adapter from LPSL	System generates provider_adapter class from LPSL	Code generation uses only fields in LPSL. No inference	P4	lpsl_codegen.py	LPSL says auth=api_key → generated code uses header X-API-Key	codegen_adapter() → validate_adapter()	Template: jinja2/templates/adapter.py.j2	SMR-2025-11-30#L1-LPSL	["hardcoded auth"]
Enforce LPSL request/response	LPSL Contract Layer	Enforce LPSL request/response	Runtime adapter validates all calls against LPSL schemas	Request/response not matching schema → E-LPSL-VIOLATION	P6	lpsl_validator.py	Response missing required field → exception	validate_request() → validate_response()	JSON Schema validation. ajv	SMR-2025-11-30#L1-LPSL	["schema drift"]

24. L1 DATA DICTIONARY LAYER

Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Define centralised data dictionary	Data Dictionary Layer	Define centralised data dictionary	System maintains a single, authoritative reference for all entities, fields, types, constraints,	Every entity and field is formally declared with PostgreSQL-compatible schema definitions, aligned to the ER model, Validation	P6	data_dictionary.py	All schema definitions are present, consistent, and database-ready	load_dictionary() → validate_against_er_model() → generate_postgres_schema()	Dictionary file: /spec/data_dictionary.json. Schema: JSON Schema Draft 2020-12. Must pass ajv validation	SMR-2025-11-30#L1-DD	["undocumented field"]

			and relationships	layer, and Legal Rule Engine layer							
Enforce data dictionary in validation	Data Dictionary Layer	Enforce data dictionary in validation	System uses the central dictionary to validate all input/output structures	Validation layer (P6) and Legal Rule Engine layer must reference the dictionary for type, constraint, and relationship checks	P6	data_dictionary_enforcer.py	Any field or relationship violation against the dictionary is rejected as dictionary_violation	enforce_dictionary_constraints() → cross_reference_validation_layer()	Enforcer runs before all P6 validation. Unknown field → E-SPEC-UNKNOWN-FIELD	SMR-2025-11-30#L1-DD	["schema drift"]

25. L1 IMPLEMENTATION BOUNDARY ENFORCEMENT LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Enforce C9 Implementation Boundary	Implementation Boundary Enforcement Layer	Enforce C9 Implementation Boundary	System rejects any generated artifact that violates C9 constraints in any row	Before file write, AST + static analysis confirms all technology, state, routing, and framework choices match C9 exactly. Absence of C9 → E-SPEC-UNDERDETERMINED	P6	boundary_enforcer.py	Generated code using Redux when C9="useState only" → E-BOUNDARY-VIOLATION	parse_c9() → scan_ast_for_violations() → block_write()	Static analysis: ruff, eslint, ts-morph. No runtime checks	SMR-2025-11-30#L1-IBE	["C9 omission"]
Enforce C11 Prohibited Implementations	Implementation Boundary Enforcement Layer	Enforce C11 Prohibited Implementations	System scans all generated code and dependencies for strings in C11 global + local lists	Any match in code, comments, or dependency tree → E-PROHIBITED-IMPL	P6	prohibition_scanner.py	Dependency "express" present when C11= ["Express"] → build fail	build_prohibition_set() → grep_codebase() → grep_dependencies()	Scan: case-insensitive. Include package.json, requirements.txt, go.mod	SMR-2025-11-30#L1-IBE	["allowlist bypass"]
Enforce Governance Coverage	Implementation Boundary Enforcement Layer	Enforce Governance Coverage	System validates that governance_bundle_hash covers every rule_id referenced in C10 across all rows	Missing coverage → E-GOV-INCOMPLETE. Build halts before generation	P2	coverage_auditor.py	C10 references SMR-2025-11-30#L1-VAL but bundle hash excludes it → E-GOV-INCOMPLETE	extract_c10_refs() → hash_bundle_contents() → compare()	Bundle file: /governance/bundle.json. Must contain full text of each referenced rule	SMR-2025-11-30#L1-IBE	["partial governance"]
Enforce Execution Mode Consistency	Implementation Boundary Enforcement Layer	Enforce Execution Mode Consistency	System validates L0 Execution Mode against repository state	execution_mode=rebuild with existing iteration rows → E-MODE-CONFLICT. execution_mode= extend with empty repo → E-MODE-CONFLICT	P7	mode_validator.py	Rebuild request on non-empty repo → deterministic rejection	check_repo_state() → compare_mode()	Repo state: git log count + iteration table count	SMR-2025-11-30#L1-IBE	["mode mismatch"]

26. L1 ACTION RESEARCH SAFETY LAYER
Columns: Row | C1 Layer | C2 Function | C3 Actor–System Interaction | C4 Full Deterministic Description | C5 DFD Mapping | C6 Code Mapping | C7 Black Box Test Case | C8 White Box Test Trace | C9 Implementation Boundary | C10 Governance Hash | C11 Prohibited Implementations

Row	C1 Layer	C2 Function	C3 Actor–System Interaction	C4 Full Deterministic Description	C5 DFD Mapping	C6 Code Mapping	C7 Black Box Test Case	C8 White Box Test Trace	C9 Implementation Boundary	C10 Governance Hash	C11 Prohibited Implementations
Enforce AR hard freeze	Action Research Safety Layer	Enforce AR hard freeze	System halts AR modifications after iteration N	If iteration_no > spec.iteration.ar_hard_freeze_after_iter, AR constraints rejected with E-AR-FROZEN	P7	ar_safety.py	Iter 6 with freeze=5 → E-AR-FROZEN	check_iter() → reject_ar_constraint()	Default ar_hard_freeze_after_iter=5	SMR-2025-11-30#L1-ARS	["unbounded AR"]
Enforce AR rollback on	Action Research Safety Layer	Enforce AR rollback on	System auto-reverts spec to last passing iteration if validation	Divergence = new E-* code not present in prior iteration	P7	ar_safety.py	Iter 4 fails E-SYNTAX, Iter 5 fails E-SYNTAX →	detect_divergence() → rollback_spec()	Rollback target: max iteration where	SMR-2025-11-30#L1-ARS	["divergence persistence"]

divergence		divergence	diverges for 2 consecutive iterations				rollback to Iter 3 spec		status='pass'		
Limit AR spec growth	Action Research Safety Layer	Limit AR spec growth	System terminates loop if spec byte size grows >10% in one iteration	Prevents constraint explosion	P7	ar_safety.py	Iter 3 spec 100KB, Iter 4 spec 115KB → E-AR-GROWTH	measure_spec_size() → compare() → terminate()	Threshold: spec.iteration.ar_max_growth_pct=10	SMR-2025-11-30#L1-ARS	["unbounded growth"]

27. EMBEDDED GOVERNANCE

27.1 To satisfy clause 0.2 and make this artefact standalone for P1–P12 execution, the minimal Supra-MetaRules v5.6 clauses required for build are embedded below. Full SMR remains canonical, but these clauses are inlined to eliminate external dependency.

27.1.1 Truth Discipline: The system must not fabricate factual content. Any speculative statement must be explicitly marked as speculation. No hidden inferential leaps are permitted. When uncertain, the system must state: “I cannot confirm this.”

27.1.2 Evidence Requirements: All factual claims must be supported by evidence or logical derivation. Citations must be inserted at the time of writing. All logical chains must be fully visible within the conversation.

27.1.3 Deterministic Mode: This is the default mode for all governance tasks, academic writing, legal analysis, marking, protocol work, and evidence-based reasoning. No probabilistic reasoning is permitted unless Heuristic Mode is explicitly activated.

27.1.4 Completeness Discipline: All reasoning must be articulated in sequential steps. No compression or omission is permitted unless the user authorises heuristic summarisation.

27.1.5 Multi-Stage Reasoning: Multi-stage reasoning must be linear and fully visible. No collapsing or skipping of reasoning stages is permitted.

27.1.6 Versioning Discipline: Any structural or behavioural modification requires a version increment. Every version of this SST must be fully reproduced in the conversation. Partial updates, deltas, or references to earlier versions are prohibited.

27.1.7 Absolute Ban on Cross-Version Referencing: The system may never reference any clause, section, or wording from a previous version of this SST. Every version must be fully self-contained, independent, and complete. Any attempt to reference a previous version constitutes a structural breach requiring immediate halt.

27.1.8 Pre-Output Self-Audit Gate: Before generating output, the system must check: compliance with all Supra-MetaRules, compliance with the invoked domain protocol, absence of unquoted abbreviations, visibility of reasoning, correct reasoning mode (Deterministic or Heuristic), compliance with the Numbered Options Rule, compliance with the Export Integrity Rule when relevant. If any issue is detected, the system must halt and self-correct.

27.2 LWP [Locked Workflow Process] is defined as: Law Map → Form Pathway → Validation. Conclusion must be non-advisory. No entitlement, liability, outcome, or prediction